TD9: Correction

CPES « Sciences des données, arts et cultures »

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

18 mars 2025

Exercice 1 On considère un digraphe D représenté par une *liste d'adjacence*, c'est à dire, où chaque sommet est associé à la liste de ses voisins sortants. Combien de temps (en notation asymptotique) faut-il pour calculer les degrés sortants de tous les sommets ? Les degrés entrants de tous les sommets ? Justifier votre résultat en détaillant les différentes étapes de votre raisonnement.

Correction : Il y a donc deux cas de figure à traiter. Les voici :

— *Degrés sortants.* Comme aucune hypothèse n'est faite sur l'implémentation des listes, nous n'avons pas la garantie que la taille de la liste est mise à jour et donc accessible en $O(1)$. De ce fait, pour connaître le degré sortant de chacun des sommets de $D = (V, E)$, il faut parcourir la liste des voisins de chacun des sommets de V. Cela a un coût total de $O(|E|)$, avec $|E|$ le nombre d'arêtes de D. Il faut également faire attention à prendre en compte de potentiels sommets qui auraient une liste d'adjacence vide, donc ne contenant aucun voisin. Vérifier qu'une liste est vide se fait aisément en $O(1)$. Ainsi, pour couvrir tous les cas de figure possibles, on peut écrire la complexité temporelle en $O(|E| + |V|)$, où $|V|$ est le nombre de sommets de D.

— *Degrés entrants.* Dans ce cas, il n'est pas possible d'accéder directement à la liste des « voisins entrants »directement pour chaque sommet. Il va falloir parcourir la liste de voisins de chaque sommet de D, et compter le nombre de fois qu'apparaît chaque sommet. Le maintient de $|V|$ compteurs se fait en $O(1)$ à chaque mise à jour, et on aura en tout $|E|$ mises à jour (une par arête). Cela fait donc une complexité temporelle également en $O(|E| + |V|)$, qui couvre aussi le cas où l'on aurait des listes de voisins vides pour tout ou partie des sommets V.

Exercice 2 Montrer qu'un sommet u ayant un degré entrant et un degré sortant strictements positifs, peut se retrouver seul dans une composante connexe de la forêt DFS. Pour rappel, la forêt DFS est définie par $D = (V, E_\pi)$, où $E_\pi = \{(u.\pi, u) : u \in V\}$.

Correction : La clé ici réside dans l'ordre de visite des sommets par DFS. Il suffit de donner un exemple pour lequel on vérifie la situation exposée. Prenons un digraphe $D = (V, E)$, avec $V = \{u, x, y\}$ et $E = \{(x, u), (u, y)\}$. Concrètement, cela correspond au digraphe suivant : $x \to u \to y$. On a bien ici un sommet u ayant un degré entrant et un degré sortant strictement positifs (valant 1 dans les deux cas). On peut dès lors procéder dans cet ordre :

— Appliquer DFS(y). Comme aucun autre sommet n'est accessible depuis y, le parcours s'arrête immédiatement. La forêt DFS (initialement vide) est alors maintenant composée d'un seul nœud y, formant de fait une composante connexe seul.

— Appliquer DFS(u). Ici le seul sommet accessible y est déjà visité, donc le parcours s'arrête aussitôt. On ajoute alors u à la forêt DFS, qui n'est de fait connecté à aucun autre nœud. On a, à ce stade, deux noeuds (y et u) qui forment chacun une composante connexe où ils sont seuls.

— Finalement, on applique DFS(x). Encore une fois, le seul somme accessible u est déjà visité, donc le parcours s'arrête immédiatement. On ajoute alors x à la forêt DFS, seul. On a ainsi trois composantes connexes qui ne contiennent qu'un nœud, que sont y, u et x. Comme tous les sommets de V ont été visités, le parcours DFS global s'arrête et on a bien réussi à isoler u dans une composante connexe de la forêt DFS.

Exercice 3 Montrer que les composantes fortement connexes d'un digraphe D forment une partition de V (l'ensemble des sommets de D), autrement dit que chaque sommet est dans une unique composante fortement connexe. On rappelle que deux sommets $x, y \in V$ sont dans la même composante fortement connexe si et seulement si il existe un chemin dirigé entre x et y et de y vers x.

Correction : Montrons que dans un digraphe $D = (V, E)$, chaque sommet $v \in V$ est dans une unique composante fortement connexe. Pour ce faire, raisonnons par l'absurde. On considère donc que v est simultanément présent dans deux composantes fortement connexes distinctes. Notons les C_1 et C_2. Cela implique donc que pour $y_1 \in C_1$, il existe un chemin dirigé entre y_1 et v et entre v et y_1. Similairement pour $y_2 \in C_2$, il existe un chemin dirigé entre y_2 et v et v et y_2.

On en déduit qu'il existe un chemin dirigé entre y_1 et y_2 et y_2 et y_1 : en effet, il suffit de suivre les chemins dirigés décrits ci-dessus qui passent par v. Un chemin dirigé existant dans les deux sens entre y_1 et y_2, alors ceux-ci sont dans une même composante fortement connexe, par définition. Ceci vient contredire le fait que les composantes fortement connexes C_1 et C_2 sont distinctes. Autrement dit, chaque v n'est présent que dans une unique composante fortement connexe de D, ce qui vient conclure la preuve.

Exercice 4 Comment le nombre de composantes fortement connexes d'un digraphe peut-il changer si un nouvel arc est ajouté ? Justifier.

Correction : Il s'agit ici d'étudier l'évolution du nombre de composantes fortement connexes d'un digraphe lorsque qu'un nouvel arc (une arête) est ajouté au digraphe.

Premièrement, le nombre de composantes fortement connexes ne peut augmenter. Pour ce faire, il faudrait retirer un certain nombre d'arêtes afin d'isoler un sous-ensemble de sommets d'une composante fortement connexe ; or ici nous ajoutons une arête au digraphe. On montre ensuite que le nombre de composantes fortement connexes peut ne pas bouger ou diminuer de façon arbitraire, en fonction du digraphe et de l'arc qui est ajouté. Considérons un digraphe constitué de n sommets, tel que, pour tout $i \in \{2, \ldots, n\}$, on ait une arête (v_i, v_{i+1}). Graphiquement, il s'agit du digraphe $v_1 \to v_2 \to \cdots \to v_n$. Ce digraphe contient n composantes fortement connexes (chaque sommet est seul dans sa composante fortement

connexe). On peut alors réduire à notre guise ce nombre de composantes en choisissant judicieusement l'arc ajouté. Ainsi, en ajoutant un arc (v_n, v_n) (sur lui-même donc), on ne change pas le nombre de composantes fortement connexes. En revanche, en ajoutant un arc (v_n, v_1), on obtient une unique composante fortement connexe, puisque qu'il existe un chemin dirigé dans les deux sens entre chaque paire de sommets. En ajoutant un arc (v_n, v_2), Les sommets $(v_2, \ldots, v_n)$ formeront une unique composante fortement connexe, alors que v_1 restera seul dans sa composante fortement connexe : il y aura donc 2 telles composantes formées dans le digraphe. Plus généralement, $\forall i \in \{1, \ldots, n\}$, ajouter un arc (v_n, v_i) résulte en la présence de i composantes fortement connexes dans le digraphe.

Exercice 5 Montrez comment s'exécute le parcours en profondeur sur le graphe de la Figure 1. Pour ce faire, repartir des algorithmes DFS et DFS-VISITE, et considérer que la boucle for de DFS (ligne 4) parcours les sommets dans leur ordre alphabétique, et que leur liste d'adjacence est donnée dans ce même ordre alphabétique. On présentera l'exécution en montrant les temps de découverte et de fin pour chaque sommet, ainsi que la forêt DFS générée.

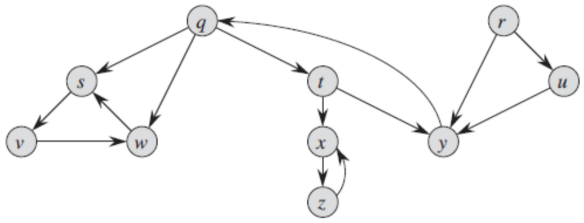


FIGURE 1 – Graphe de la question 5.

Correction : Exécutons l'algorithme pas à pas, en utilisant le pseudo-code des algorithmes DFS et DFS-VISITE. On commence dans DFS par initialiser π et la couleur de chaque sommet à leur valeur par défaut. La variable temps est définie à 1. On entre ensuite dans la boucle pour de DFS, en suivant l'ordre alphabétique des sommets. Voici l'exécution pas à pas :

1. DFS-VISITE(D, q) : q.couleur prend gris. $q.d$ prend 1 et temps vaut 2. La liste d'adjacence de q est $[s, t, w]$. On la parcourt (dans l'ordre alphabétique des sommets) : on commence par s. $s.\pi$ prend q, puis appel à DFS-VISITE(D, s).
2. DFS-VISITE(D, s) : s.couleur prend gris. $s.d$ prend 2 et temps vaut 3. La liste d'adjacence de s est $[v]$. On la parcourt, commençant (et finissant par) v. $v.\pi$ prend s, puis appel à DFS-VISITE(D, v).
3. DFS-VISITE(D, v) : v.couleur prend gris. $v.d$ prend 3 et temps vaut 4. La liste d'adjacence de v est $[w]$. On la parcourt, commençant (et finissant) par w. $w.\pi$ prend v, puis appel à DFS-VISITE(D, w).
4. DFS-VISITE(D, w) : w.couleur prend gris. $w.d$ prend 4 et temps vaut 5. La liste d'adjacence de w est $[s]$. On la parcourt, commençant (et finissant) par s. Mais s.couleur vaut gris, donc la boucle se termine aussitôt. w.couleur prend noir, $w.f$ prend 5 et temps prend 6. On retourne à l'appel DFS-VISITE(D, v).

5. Retour à DFS-VISITE(D, v) : w est traité, on doit passer au voisin suivant. Mais w était seul voisin de v, donc la boucle se termine. v.couleur prend noir, $v.f$ prend 6 et temps prend 7. Retour à l'appel de DFS-VISITE(D, s).
6. Retour à DFS-VISITE(D, s) : v est traité, on doit passer au voisin suivant. Mais v était seul voisin de s, donc la boucle se termine. s.couleur prend noir, $s.f$ prend 7 et temps prend 8. Retour à l'appel de DFS-VISITE(D, q).
7. Retour à DFS-VISITE(D, q) : s est traité, on doit passer au voisin suivant, qui est t. t.couleur vaut blanc, donc on peut le traiter : $t.\pi$ prend q, et on appelle DFS-VISITE(D, t).
8. DFS-VISITE(D, t) : t.couleur prend gris. $t.d$ prend 8 et temps vaut 9. La liste d'adjacence de t est $[x, y]$. On la parcourt, commençant par x, qui est blanc. $x.\pi$ prend t, puis appel à DFS-VISITE(D, x).
9. DFS-VISITE(D, x) : x.couleur prend gris. $x.d$ prend 9 et temps vaut 10. La liste d'adjacence de x est $[z]$. On la parcourt, commençant (et finissant) par z, qui est blanc. $z.\pi$ prend x, puis appel à DFS-VISITE(D, z).
10. DFS-VISITE(D, z). z.couleur prend gris. $z.d$ prend 10 et temps vaut 11. La liste d'adjacence de z est $[x]$. On la parcourt, commençant (et finissant) par z. Mais x est gris à ce stade, donc la boucle s'arrête instantanément. z.couleur prend noir, $z.f$ prend 11 et temps prend 12. Retour à l'appel de DFS-VISITE(D, x).
11. Retour à DFS-VISITE(D, x) : z est traité. Il était le seul voisin de x, donc la boucle prend fin. x.couleur prend noir, $x.f$ prend 12 et temps prend 13. Retour à l'appel DFS-VISITE(D, t).
12. Retour à DFS-VISITE(D, t) : x est traité, on passe au voisin suivant qui est y. Il est blanc, et donc $y.\pi$ prend t, puis appel à DFS-VISITE(D, y).
13. DFS-VISITE(D, y) : y.couleur prend gris. $y.d$ prend 13 et temps prend 14. La liste d'adjacence de y est $[q]$. On la parcourt, commençant (et finissant) par q. Mais q est gris, donc la boucle s'arrête immédiatement. y.couleur prend noir, $y.f$ prend 14 et temps prend 15. Retour à l'appel DFS-VISITE(D, t).
14. Retour à DFS-VISITE(D, t) : y est traité, qui était le dernier voisin de t. La boucle prend fin, t.couleur prend noir, $t.f$ prend 15 et temps prend 16. Retour à DFS-VISITE(D, q).
15. Retour à DFS-VISITE(D, q) : t est traité, on doit passer au voisin suivant, qui est w. Mais w est noir, donc il n'est pas traité et la boucle se termine. On a alors q.couleur prend noir, $q.f$ prend 15 et temps prend 16. Ceci vient terminer l'appel DFS-VISITE(D, q), et on retourne dans DFS(D).
16. Retour à DFS(D) : on continue la boucle principale de DFS. L'itération suivante est r, qui est blanc. On appelle donc DFS-VISITE(D, r).
17. DFS-VISITE(D, r) : r.couleur prend gris, $r.d$ prend 16 et temps prend 17. La liste d'adjacence de r est $[u, y]$. On la parcourt en commençant par u qui est blanc. $u.\pi$ prend r, puis appel à DFS-VISITE(D, u).
18. DFS-VISITE(D, u) : u.couleur prend gris, $u.d$ prend 17 et temps prend 18. La liste d'adjacence de u est $[y]$. Mais y est déjà noir, donc la boucle s'arrête instantanément. u.couleur prend noir, $u.f$ prend 18 et temps prend 19. Retour à l'appel DFS-VISITE(D, r).

19. Retour à DFS-VISITE(D, r) : u est traité. Le voisin restant de r est y mais qui est déjà noir : la boucle s'arrête. r.couleur prend noir, $r.f$ prend 19 et temps prend 20. Retour à DFS(D).
20. Retour à DFS(D) : on continue les itérations de la boucle pour. Il reste dans cette boucle $[s, t, u, v, w, x, y, z]$. Tous ces sommets sont à noir, et donc on termine l'exécution de la boucle pour, terminant ainsi le parcours en largeur.

Les temps de découverte et de fin sont synthétisés dans le Tableau 1. La forêt DFS générée par le parcours est présentée en Figure 2.

Sommet	q	r	s	t	u	v	w	x	y	z
Découverte	1	16	2	8	17	3	4	9	13	10
Fin	15	19	7	15	18	6	5	12	14	11

TABLE 1 – Temps de découverte et de fin lors du parcours DFS du graphe de la Figure 1

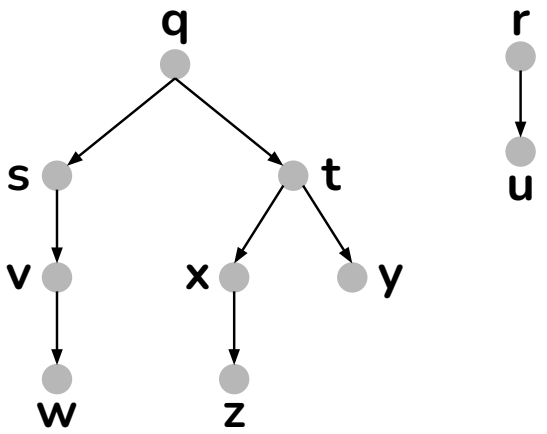


FIGURE 2 – Forêt DFS générée par le parcours du graphe de la Figure 1